

L00 · CORALNPU 教学仓

# L00 · 环境、工具链与仓库地图

作业说明 —— 任务拆解、验收标准与调试方法

---

对应上游仓库: [google/coralnpu](https://github.com/google/coralnpu) (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → `./learn check` 对拍 → 修复

文档版本: 2026-09-13 · 生成命令: `./learn pdf`

# 目录

---

## L00 作业说明

### 0. 交付物

作业 1：环境自检

作业 2：仓库地图问答（5 题）

作业 3：你的第一个 RTL 模块（alu32.sv）

作业 4：端到端跑通一个真实程序

验收

常见坑

学习记录（建议提交）

# L00 作业说明

## 0. 交付物

如果下面的术语（交叉编译、ELF、RTL、组合逻辑、寄存器）有看不懂的，先跑 `./learn diag` 定位缺哪一层，再上预备课 P0： `./learn start P0`。P0 与本节作业一一对应，通常 1 小时内能补完。

| 交付物             | 位置                                                 | 通过标准                           |
|-----------------|----------------------------------------------------|--------------------------------|
| 作业 1 环境         | 无文件，跑命令                                            | <code>./learn doctor</code> 全绿 |
| 作业 2 仓库地图       | <code>course/lessons/L00_setup/answers.md</code>   | 5 个问题全对                        |
| 作业 3 第一个 RTL 模块 | <code>course/lessons/L00_setup/rtl/alu32.sv</code> | 2360 组向量全对                     |
| 作业 4 端到端跑通      | 无文件，跑命令                                            | C 程序在黄金模型上跑出正确结果               |

一条命令检查全部：

```
./learn check L00
```

## 作业 1：环境自检

```
./learn doctor
```

预期看到每一项都是（`verilator` 是可选，L10 才用得到）：

```
python3          3.12.x
iverilog         Icarus Verilog version 12.0
riscv64-unknown-elf-gcc 13.2.0
pandoc / weasyprint 用于渲染课程 PDF
中文字体        Noto Sans CJK
```

缺东西就装：

```
./learn doctor --install
```

**为什么要先跑这一条？** 硬件开发最常见的挫败感来自「环境不对」而不是「不会写」。把环境的判断交给工具，你只需要专注设计本身。

## 作业 2：仓库地图问答（5 题）

打开 `course/lessons/L00_setup/answers.md`，把 5 个 `TODO` 换成你的答案。

题目与线索：

| 题号 | 问题                                | 去哪里找                                                                              |
|----|-----------------------------------|-----------------------------------------------------------------------------------|
| Q1 | 上游三档核心的 Bazel 目标名前缀（标量 / 向量 / 矩阵） | <code>hdl/chisel/src/coralnpu/BUILD</code> 搜 <code>core_mini</code>               |
| Q2 | ITCM 与 DTCM 的起始地址和默认大小            | <code>doc/integration_guide.md</code> 的「CoralNPU Memory Map」                      |
| Q3 | 上游用哪条指令停机？机器码是多少？                 | <code>toolchain/crt/coralnpu_start.S</code> 结尾，或 <code>scalar/Decode.scala</code> |
| Q4 | 向量后端的 SystemVerilog 源码在哪个目录？      | <code>ls hdl/verilog/</code>                                                      |
| Q5 | 上游最基础的 cocotb 测试用哪条命令跑？           | 根目录 <code>README.md</code> 的 Quick Start                                          |

这一题的目的不是考记忆，而是让你知道遇到问题时该去仓库的哪个角落找答案——这比记住答案重要得多。

## 作业 3：你的第一个 RTL 模块（alu32.sv）

### 规格

打开 `course/lessons/L00_setup/rtl/alu32.sv`，你会看到 3 个 `TODO`。

| op | 运算   | 提示                                                       |
|----|------|----------------------------------------------------------|
| 0  | ADD  | 已给                                                       |
| 1  | SUB  | <code>a - b</code>                                       |
| 2  | SLL  | <code>a &lt;&lt; b[4:0]</code>                           |
| 3  | SLT  | 有符号比较： <code>\$signed(a) &lt; \$signed(b)</code> ，结果 0/1 |
| 4  | SLTU | 无符号比较： <code>a &lt; b</code> ，结果 0/1                     |
| 5  | XOR  |                                                          |
| 6  | SRL  | <code>a &gt;&gt; b[4:0]</code>                           |
| 7  | SRA  | 需要先写 <code>sra_result</code> （ <code>TODO 1</code> ）     |
| 8  | OR   |                                                          |
| 9  | AND  | <code>default</code> 分支里                                 |

## 步骤

1. 先只填 `sra_result` 与 `zero`，跑一次 `./learn check L00`，看失败信息；
2. 再补 `case` 里剩下的分支；
3. 直到出现 2360 组向量全部一致。

## 技术点

**算术右移：** `$signed(a) >>> b` 在无符号上下文里会退化成逻辑右移。稳妥写法是先逻辑右移，再把高位填成符号位：

```
sra_result = (a >> shamt) | ({32{a[31]}} & ~(32'hFFFF_FFFF >> shamt));
```

**比较有符号性：** SLT 和 SLTU 的区别就是「把最高位当符号位还是当数值位」。

## 作业 4：端到端跑通一个真实程序

这一步让你第一次看到完整的链路：**C 代码 → 交叉编译 → ELF → 内存镜像 → 执行 → 核对结果。**

```
# 1. 编译 C 程序（会生成 ELF、program.hex、data.hex、symbols.json）
python3 course/tools/build_program.py \
    course/lessons/L00_setup/tests/programs/hello.c \
    --out-dir course/work/L00/hello

# 2. 在黄金模型上运行它，并读回结果
python3 - <<'PY'
import json, sys
sys.path.insert(0, 'course')
from golden.rv32i import Core

core = Core()
core.load_elf('course/work/L00/hello/hello.elf')
trace = core.run()
print('停机原因:', core.halt_reason, '指令数:', len(trace))

sym = json.load(open('course/work/L00/hello/symbols.json'))
for name in ('answer', 'steps'):
    off = sym[name] - 0x10000
    print(name, '=', int.from_bytes(core.dtcmm[off:off+4], 'little'))
PY
```

预期输出：

```
停机原因: mpause 指令数: 133
answer = 45
steps = 9
```

然后观察前几行 trace，理解「每条指令一行」是什么意思：

```
python3 - <<'PY'
import sys; sys.path.insert(0, 'course')
from golden.rv32i import Core
from tools.disasm import disasm
core = Core(); core.load_elf('course/work/L00/hello/hello.elf')
for i, r in enumerate(core.run()):
    if i >= 12: break
    print(f"{r.pc:08x} {disasm(r.inst):24s} rd={r.rd:2d} wdata=0x{r.wdata:08x}")
PY
```

## 验收

./learn check L00

期望：

[通过] 作业1 环境自检  
 [通过] 作业2 仓库地图  
 [通过] 作业3 alu32.sv            2360 组向量全部一致  
 [通过] 作业4 端到端跑通        answer=45 steps=9

4/4 个作业通过

## 常见坑

| 现象                          | 原因                       | 修法                          |
|-----------------------------|--------------------------|-----------------------------|
| iverilog 编译失败               | 语法错误或模块名/端口名写错           | 看报错行号；模块必须叫 alu32           |
| zero 标志不对                   | 忘了给 zero 赋值，或写成 y = 0    | assign zero = (y == 32'h0); |
| 某些向量在 X/F 之间跳动              | case 有分支没赋值              | 补 default，并给默认值             |
| 作业 2 一直不通过                  | 答案写得不完整（例如只写了一个地址）       | 每条答案都覆盖题目要求的全部要点            |
| 作业 4 报符号表找不到                | 变量名写错或没用 volatile 导致被优化掉 | 用课程提供的 hello.c 原样编译         |
| 找不到 riscv64-unknown-elf-gcc | 工具链没装                    | ./learn doctor --install    |

## 学习记录（建议提交）

在 course/work/L00/记录.md 里写下三件事：

1. 你的机器上装工具链花了多久、遇到什么问题；
2. 作业 3 里你第一次跑 ./learn check L00 时的失败信息是什么，你怎么修的；
3. 作业 4 的 trace 里，第一条 auipc 指令的 wdata 是多少？为什么是这个值（提示：它是当前 PC 加一个高位立即数）。